

Федеральное государственное автономное образовательное учреждение
высшего образования

**«ОМСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Кафедра Автоматизированные системы обработки информации и
управления.

Лабораторная работа №2

по дисциплине «Основы профессиональной деятельности» на тему:

«Разработка телеграм-бота.»

Выполнил: студент группы ИВТ-
244 Шмидт Антон Владиславович
Проверил: ассистент М.А.
Семибратов

Омск 2025

Введение

Telegram-бот — это автоматизированное программное приложение, разработанное для работы в мессенджере Telegram, которое взаимодействует с пользователями через текстовые команды, сообщения или встроенные интерфейсы (например, кнопки). Бот функционирует на основе API Telegram, позволяющего ему отправлять и получать сообщения, обрабатывать запросы и выполнять заданные действия, такие как предоставление информации, выполнение задач или интеграция с внешними сервисами. Telegram-боты создаются с использованием различных языков программирования (например, Python, JavaScript) и могут быть настроены для индивидуального или группового общения, что делает их универсальным инструментом для автоматизации процессов и взаимодействия с аудиторией.

Задача

Чат-бот записи в больницу (ФИО пациента, выбор врача, выбор времени и даты).

Ход работы

1. Создаём новый проект в PyCharm, инициализируем в нём Git и привязываем новый GitHub репозиторий.
2. Получаем токен бота в BotFather в телеграм.
3. Создаём структуру проекта и файл .env, где указываем наш токен.

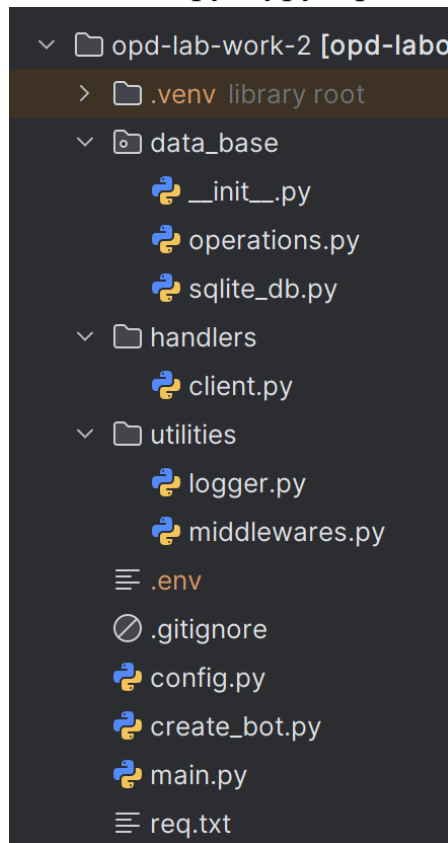


Рисунок 1. Структура проекта

4. Пишем функционал базы данных, мидделваря и логгера.
5. Прописываем нужные нам обработчики и FSM машину.
6. Проверяем бота в телеграм.
7. Загружаем проект на GitHub.

Код

```
main.py x
1  # Python модули
2  from aiogram.types.bot_command import BotCommand
3
4  import asyncio
5  import nest_asyncio
6  import platform
7
8
9  # Локальные модули
10 from create_bot import bot, dp, db
11 from handlers import client
12 from utilities.logger import logger
13 from utilities.middlewares import StandardMiddleware
14
15
16 # Функции при запуске и выключении бота
17 async def on_startup(): 1 usage  1 pplx
18     nest_asyncio.apply()
19
20     await bot.set_my_commands(commands=[
21         BotCommand(command='/start', description='Запустить/Перезапустить бота'),
22     ])
23
24     bot_user = await bot.get_me()
25
26     if await db.connect():
27         logger.info(f'BOT_NAME="{bot_user.full_name}", BOT_USERNAME="{bot_user.username}", MESSAGE="database started"')
28     else:
29         logger.error(f'BOT_NAME="{bot_user.full_name}", BOT_USERNAME="{bot_user.username}", MESSAGE="database NOT started"')
30
31     logger.info(f'BOT_NAME="{bot_user.full_name}", BOT_USERNAME="{bot_user.username}", MESSAGE="up and running..."')
```

Рисунок 2. Код main.py 1

```
main.py x
34 async def on_shutdown(): 1 usage  1 pplx
35     bot_user = await bot.get_me()
36
37     if await db.close():
38         logger.info(f'BOT_NAME="{bot_user.full_name}", BOT_USERNAME="{bot_user.username}", MESSAGE="database finished"')
39     else:
40         logger.error(f'BOT_NAME="{bot_user.full_name}", BOT_USERNAME="{bot_user.username}", MESSAGE="database NOT finished correct"')
41
42     logger.info(f'BOT_NAME="{bot_user.full_name}", BOT_USERNAME="{bot_user.username}", MESSAGE="shutting down..."')
43
44
45 # Функция запуска бота
46 async def main(): 1 usage  1 pplx
47     dp.startup.register(on_startup)
48     dp.shutdown.register(on_shutdown)
49
50     dp.include_routers(
51         client.router
52     )
53
54     dp.message.middleware(StandardMiddleware())
55     dp.callback_query.middleware(StandardMiddleware())
56
57     await bot.delete_webhook(drop_pending_updates=True)
58     await dp.start_polling(bot)
59
60
61 # Запуск бота
62 if __name__ == '__main__':
63     try:
64         if platform.system() == 'Windows':
65             asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
66
67             asyncio.run(main())
68     except KeyboardInterrupt:
69         asyncio.run(bot.session.close())
```

Рисунок 3. Код main.py 2

```

create_bot.py x
1  # Python модули
2  from aiogram import Bot, Dispatcher
3  from aiogram.client.bot import DefaultBotProperties
4  from aiogram.fsm.storage.memory import MemoryStorage
5
6
7  # Локальные модули
8  import config
9
10 from data_base.sqlite_db import SQLiteDB
11
12
13 # Основные объекты для взаимодействия
14 def_props = DefaultBotProperties(
15     parse_mode='HTML',
16     link_preview_is_disabled=True
17 )
18
19 bot = Bot(
20     token=config.BOT_TOKEN,
21     default=def_props
22 )
23
24 dp = Dispatcher(
25     storage=MemoryStorage()
26 )
27
28 db = SQLiteDB(
29     db_name='base.db'
30 )

```

Рисунок 4. Код create_bot.py

```

config.py x
1  # Python модули
2  from dotenv import load_dotenv
3
4  import os
5
6
7  # Чтение переменных окружения из .env файла
8  load_dotenv(override=True)
9
10 BOT_TOKEN = os.environ['BOT_TOKEN']

```

Рисунок 5. Код config.py

```

__init__.py x
1  from data_base import sqlite_db

```

Рисунок 6. Код data_base/__init__.py

```
sqlite_db.py x
1  # Python модули
2  import aiosqlite
3
4
5  # Локальные модули
6  from utilities.logger import logger
7
8
9  # Классы
10 class SQLiteDB: 12 usages  ppljc
11     def __init__(self, db_name):  ppljc
12         self.db_name = db_name
13         self.connection = None
14
15     async def connect(self): 1 usage (1 dynamic)  ppljc
16         try:
17             self.connection = await aiosqlite.connect(self.db_name)
18             await self.connection.execute(
19                 '''
20                 CREATE TABLE IF NOT EXISTS users
21                 (date DATE, user_id INTEGER UNIQUE, first_name TEXT, last_name TEXT,
22                  username TEXT UNIQUE)
23                 '''
24             )
25             await self.connection.execute(
26                 '''
27                 CREATE TABLE IF NOT EXISTS appointments
28                 (user_id INTEGER, date DATE, time TEXT, doctor_area TEXT, doctor_username TEXT)
29                 '''
30             )
31             await self.connection.execute(
32                 '''
33                 CREATE TABLE IF NOT EXISTS doctor_areas
34                 (doctor_area TEXT, doctor_area_name TEXT)
35                 '''
36             )
```

Рисунок 7. Код data_base/sqlite_db.py 1

```
sqlite_db.py x
10 class SQLiteDB: 12 usages 1 ppljc
15     async def connect(self): 1 usage (1 dynamic) 1 ppljc
37         await self.connection.execute(
38             '''
39             CREATE TABLE IF NOT EXISTS doctors
40             (doctor_username TEXT UNIQUE, doctor_full_name TEXT, doctor_area TEXT)
41             '''
42         )
43         await self.connection.commit()
44         return True
45     except Exception as e:
46         logger.error(f'USER=BOT, MESSAGE="{e}"')
47         return False
48
49     async def close(self): 3 usages (3 dynamic) 1 ppljc
50         try:
51             await self.connection.close()
52             return True
53         except Exception as e:
54             logger.error(f'USER=BOT, MESSAGE="{e}"')
55             return False
```

Рисунок 8. Код data_base/sqlite_db.py 2

```
sqlite_db.py x
10 class SQLiteDB: 12 usages 1 ppljc
57     async def insert(self, table: str, what: dict) -> bool: 2 usages 1 ppljc
58         try:
59             columns = ','.join(what.keys())
60             placeholders = ','.join(['?'] * len(what))
61             values = tuple(what.values())
62
63             query = f'''INSERT INTO {table} ({columns}) VALUES ({placeholders})'''
64             await self.connection.execute(query, values)
65             await self.connection.commit()
66
67             logger.debug(f'USER=BOT, MESSAGE="table={table}, what={what}"')
68             return True
69         except aiosqlite.IntegrityError:
70             logger.debug(f'USER=BOT, MESSAGE="UNIQUE ALREADY EXISTS | table={table}, what={what}"')
71             return False
72         except Exception as e:
73             logger.error(f'USER=BOT, MESSAGE="{e}"')
74             return False
```

Рисунок 9. Код data_base/sqlite_db.py 3

```

sqlite_db.py x
10 class SQLiteDB: 12 usages ppljc
99     async def update(self, table: str, what: dict, where: dict | None = None): ppljc
100         try:
101             what_clause = ', '.join([f'{col} = ?' for col in what.keys()])
102             what_values = tuple(what.values())
103
104             if where:
105                 where_clause = ' AND '.join([f'{col} = ?' for col in where.keys()])
106                 where_values = tuple(where.values())
107                 query = f'''UPDATE {table} SET {what_clause} WHERE {where_clause}'''
108                 values = what_values + where_values
109             else:
110                 query = f'''UPDATE {table} SET {what_clause}'''
111                 values = what_values
112
113             cursor = await self.connection.execute(query, values)
114             await self.connection.commit()
115
116             affected_rows = cursor.rowcount
117             logger.debug(
118                 f'USER=BOT, MESSAGE="table={table}, what={what}, where={where}, '
119                 f'affected_rows={affected_rows}"'
120             )
121             return affected_rows
122         except Exception as e:
123             logger.error(f'USER=BOT, MESSAGE="{e}"')
124             return 0

```

Рисунок 10. Код data_base/sqlite_db.py 4

```

sqlite_db.py x
10 class SQLiteDB: 12 usages ppljc
76     async def select(self, table: str, what: tuple | None = None, where: dict | None = None):
77         try:
78             what_clause = '*' if not what else ', '.join(what)
79
80             if where:
81                 where_clause = ' AND '.join([f'{key} = ?' for key in where.keys()])
82                 where_values = tuple(where.values())
83                 query = f'''SELECT {what_clause} FROM {table} WHERE {where_clause}'''
84                 cursor = await self.connection.execute(query, where_values)
85             else:
86                 query = f'''SELECT {what_clause} FROM {table}'''
87                 cursor = await self.connection.execute(query)
88
89             rows = await cursor.fetchall()
90             logger.debug(
91                 f'USER=BOT, MESSAGE="table={table}, what={what_clause}, where={where}, '
92                 f'result_len={len(rows)}"'
93             )
94             return rows
95         except Exception as e:
96             logger.error(f'USER=BOT, MESSAGE="{e}"')
97             return []

```

Рисунок 11. Код data_base/sqlite_db.py 5

```

sqlite_db.py x
10 class SQLiteDB: 12 usages 1 ppljc
99     async def update(self, table: str, what: dict, where: dict | None = None): 1 ppljc
117         logger.debug(
118             f'USER=BOT, MESSAGE="table={table}, what={what}, where={where}, '
119             f'affected_rows={affected_rows}"'
120         )
121         return affected_rows
122     except Exception as e:
123         logger.error(f'USER=BOT, MESSAGE="{e}"')
124         return 0
125
126     async def remove(self, table: str, where: dict | None = None) -> int: 1 usage 1 ppljc
127     try:
128         if where:
129             where_clause = ' AND '.join([f'{key} = ?' for key in where.keys()])
130             where_values = tuple(where.values())
131             query = f'DELETE FROM {table} WHERE {where_clause}'
132             cursor = await self.connection.execute(query, where_values)
133         else:
134             query = f'DELETE FROM {table}'
135             cursor = await self.connection.execute(query)
136
137         await self.connection.commit()
138         affected_rows = cursor.rowcount
139         logger.debug(
140             f'USER=BOT, MESSAGE="table={table}, where={where}, affected_rows={affected_rows}"'
141         )
142         return affected_rows
143     except Exception as e:
144         logger.error(f'USER=BOT, MESSAGE="{e}"')
145         return 0

```

Рисунок 12. Код data_base/sqlite_db.py 6

```

operations.py x
1 # Python модули
2 from aiogram.utils.keyboard import InlineKeyboardBuilder
3
4 import datetime
5
6
7 # Локальные модули
8 from data_base.sqlite_db import SQLiteDB
9
10
11 # Функции
12 async def user_add(db: SQLiteDB, user): 2 usages 1 ppljc
13     date = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
14     await db.insert(
15         table='users',
16         what={
17             'date': date,
18             'user_id': user.id,
19             'first_name': user.first_name,
20             'last_name': user.last_name,
21             'username': user.username
22         }
23     )
24
25
26 async def doctor_area_name_get(db: SQLiteDB, doctor_area): 5 usages 1 ppljc
27     doctor_area_name = await db.select(
28         table='doctor_areas',
29         what=('doctor_area_name',),
30         where={'doctor_area': doctor_area}
31     )
32
33     return doctor_area_name[0][0]

```

Рисунок 13. Код data_base/operations.py 1

```

operations.py x
36 async def doctor_areas_get(db: SQLiteDB): 3 usages 1 ppjlc
37     doctor_areas = await db.select(
38         table='doctor_areas'
39     )
40
41     builder = InlineKeyboardBuilder()
42
43     for doctor in doctor_areas:
44         builder.button(text=doctor[1], callback_data=f'appointment_doctor_area|{doctor[0]}')
45
46     builder.button(text='Назад 🏠', callback_data='add_appointment')
47     builder.adjust(*sizes: 3, repeat=True)
48
49     return builder.as_markup()
50
51
52 async def doctors_get(db: SQLiteDB, doctor_area): 2 usages 1 ppjlc
53     doctors = await db.select(
54         table='doctors',
55         where={'doctor_area': doctor_area}
56     )
57
58     doctor_area_name = await doctor_area_name_get(db=db, doctor_area=doctor_area)
59
60     builder = InlineKeyboardBuilder()
61
62     for doctor in doctors:
63         builder.button(text=doctor[1], callback_data=f'appointment_doctor_username|{doctor[0]}')
64
65     builder.button(text='Назад 🏠', callback_data='appointment_full_name')
66     builder.adjust(*sizes: 1, repeat=True)
67
68     return builder.as_markup(), doctor_area_name

```

Рисунок 14. Код data_base/operations.py 2

```

operations.py x
71 async def dates_get(db: SQLiteDB, doctor_area, doctor_username): 2 usages 1 ppjlc
72     doctor_full_name = await db.select(
73         table='doctors',
74         what=('doctor_full_name',),
75         where={'doctor_username': doctor_username}
76     )
77
78     existing_appointments = await db.select(
79         table='appointments',
80         where={'doctor_area': doctor_area, 'doctor_username': doctor_username}
81     )
82
83     occupied_slots = set()
84     for appt in existing_appointments:
85         occupied_slots.add((appt[1], appt[2]))
86
87     current_datetime = datetime.datetime.now()
88     current_date = current_datetime.date()
89     available_slots = {}
90
91     for day_offset in range(7):
92         target_date = current_date + datetime.timedelta(days=day_offset)
93
94         work_start = datetime.time(hour=9, minute=0)
95         work_end = datetime.time(hour=16, minute=30)
96
97         if target_date == current_date:
98             current_time = current_datetime.time()
99             if current_time >= work_end:
100                 continue
101
102             current_minute = current_time.minute
103             remainder = current_minute % 30
104             minutes_to_add = 30 - remainder if remainder != 0 else 0
105             next_slot_time = (datetime.datetime.combine(datetime.date.today(), current_time) +
106                             datetime.timedelta(minutes=minutes_to_add)).time()

```

Рисунок 15. Код data_base/operations.py 3

```
operations.py x
71  async def dates_get(db: SQLiteDB, doctor_area, doctor_username): 2 usages  ± ppjlc *
103      remainder = current_minute % 30
104      minutes_to_add = 30 - remainder if remainder != 0 else 0
105      next_slot_time = (datetime.datetime.combine(datetime.date.today(), current_time) +
106                      datetime.timedelta(minutes=minutes_to_add)).time()
107
108      if next_slot_time >= work_end:
109          continue
110      start_time = next_slot_time
111  else:
112      start_time = work_start
113
114      start_datetime = datetime.datetime.combine(target_date, start_time)
115      end_datetime = datetime.datetime.combine(target_date, work_end)
116
117      slot = start_datetime
118      while slot <= end_datetime:
119          slot_date_str = slot.date().isoformat()
120          slot_time_str = slot.time().strftime("%H:%M")
121
122          if (slot_date_str, slot_time_str) not in occupied_slots:
123              if target_date not in available_slots:
124                  available_slots[target_date] = []
125                  available_slots[target_date].append(slot_time_str)
126
127          slot += datetime.timedelta(minutes=30)
128
129      builder = InlineKeyboardBuilder()
130      builders_date = {}
131
132      for date in sorted(available_slots.keys()):
133          builder.button(
134              text=f'{date.strftime("%d.%m.%Y")} ({len(available_slots[date])}',
135              callback_data=f'appointment_date|{date.strftime("%d.%m.%Y")}'
136          )
```

Рисунок 16. Код data_base/operations.py 4

```

operations.py x
71  async def dates_get(db: SQLiteDB, doctor_area, doctor_username): 2 usages  ppjlc *
130      builders_date = {}
131
132      for date in sorted(available_slots.keys()):
133          builder.button(
134              text=f'{date.strftime("%d.%m.%Y")} ({len(available_slots[date])}',
135              callback_data=f'appointment_date|{date.strftime("%d.%m.%Y")}'
136          )
137
138          builder_date = InlineKeyboardBuilder()
139          for time in sorted(available_slots[date]):
140              builder_date.button(text=f'{time}', callback_data=f'appointment_time|{date.strftime("%d.%m.%Y")}|{time}')
141
142          builder_date.button(text='Назад 🏠', callback_data=f'appointment_doctor_username|{doctor_username}')
143          builder_date.adjust(*sizes: 3, repeat=True)
144
145          builders_date[date.strftime("%d.%m.%Y")] = builder_date.as_markup()
146
147      builder.button(text='Назад 🏠', callback_data=f'appointment_doctor_area|{doctor_area}')
148      builder.adjust(*sizes: 2, repeat=True)
149
150      return builder.as_markup(), builders_date, doctor_full_name[0][0]
151
152
153  async def appointment_add(db: SQLiteDB, user_id, date, time, doctor_area, doctor_username): 2 usages  ppjlc
154      await db.insert(
155          table='appointments',
156          what={
157              'user_id': user_id,
158              'date': date,
159              'time': time,
160              'doctor_area': doctor_area,
161              'doctor_username': doctor_username
162          }
163      )

```

Рисунок 17. Код data_base/operations.py 5

```

operations.py x
166  async def appointments_get(db: SQLiteDB, user_id): 2 usages  ppjlc *
167      appointments = await db.select(
168          table='appointments',
169          where={'user_id': user_id}
170      )
171
172      text = ' 📄 Вот список ваших записей:'
173      builder = InlineKeyboardBuilder()
174
175      for appointment in appointments:
176          doctor_area_name = await doctor_area_name_get(db=db, doctor_area=appointment[3])
177
178          builder.button(
179              text=f'{appointment[1]} {appointment[2]} {doctor_area_name}',
180              callback_data=f'edit_appointment|{appointment[1]}|{appointment[2]}|{appointment[3]}|{appointment[4]}'
181          )
182
183      if not appointments:
184          text = ' 🙄 У вас пока нет записей.'
185
186      builder.button(text='Назад 🏠', callback_data='menu_main')
187      builder.adjust(*sizes: 1, repeat=True)
188
189      return text, builder.as_markup()

```

Рисунок 18. Код data_base/operations.py 6

```

operations.py x
189     return text, builder.as_markup()
190
191
192 async def appointment_get(db: SQLiteDB, date, time, doctor_area, doctor_username): 2 usages 1 ppjlc
193     doctor_area_name = await doctor_area_name_get(db=db, doctor_area=doctor_area)
194
195     doctor_full_name = await db.select(
196         table='doctors',
197         what=('doctor_full_name',),
198         where={'doctor_username': doctor_username}
199     )
200
201     text = (
202         f'Ваша запись на {date} {time}:\n\n'
203         f'Направление: {doctor_area_name}\n'
204         f'Доктор: {doctor_full_name[0][0]}'
205     )
206
207     builder = InlineKeyboardBuilder()
208     builder.button(text='Отменить ❌', callback_data=f'remove_appointment|{date}|{time}|{doctor_area}|{doctor_username}')
209     builder.button(text='Назад ⬅️', callback_data=f'list_appointments')
210     builder.adjust(2)
211
212     return text, builder.as_markup()
213
214
215 async def appointment_remove(db: SQLiteDB, user_id, date, time, doctor_area, doctor_username): 2 usages 1 ppjlc
216     await db.remove(
217         table='appointments',
218         where={'user_id': user_id, 'date': date, 'time': time, 'doctor_area': doctor_area, 'doctor_username': doctor_username}
219     )

```

Рисунок 19. Код data_base/operations.py 7

```

logger.py x
1  # Python модули
2  from logging.handlers import TimedRotatingFileHandler
3
4  import logging
5  import os
6
7
8  # Функционал
9  if not os.path.exists('./logs'):
10     os.makedirs('./logs')
11
12  logger = logging.getLogger('base')
13  logger.setLevel(logging.DEBUG)
14
15  file_handler = TimedRotatingFileHandler(
16     filename='./logs/base.log',
17     when="midnight",
18     interval=1,
19     encoding='utf-8'
20 )
21
22  console_handler = logging.StreamHandler()
23
24  formatter = logging.Formatter(
25     fmt='[%asctime)s] %(levelname)s [%filename)s; %(funcName)s]: %(message)s;',
26     datefmt='%H:%M:%S'
27 )
28
29  file_handler.setFormatter(formatter)
30  console_handler.setFormatter(formatter)
31
32  logger.addHandler(file_handler)
33  logger.addHandler(console_handler)

```

Рисунок 20. Код utilities/logger.py

```

middlewares.py x
1  # Python модули
2  from aiogram import BaseMiddleware
3  from aiogram.types import *
4  from typing import *
5
6
7  # Локальные модули
8  from create_bot import db
9  from data_base.operations import user_add
10
11
12 # Классы
13 class StandardMiddleware(BaseMiddleware): 3 usages  ppjlc
14     async def __call__(  ppjlc
15         self,
16         handler: Callable[[TelegramObject, Dict[str, Any]], Awaitable[Any]],
17         event: Message | CallbackQuery,
18         data: Dict[str, Any]
19     ) -> Any:
20         user = event.from_user
21         data['user'] = user
22         await user_add(db=db, user=user)
23         return await handler(event, data)

```

Рисунок 21. Код utilities/middlewares.py

```

client.py x
1  # Python модули
2  from aiogram import Router, F
3  from aiogram.fsm.context import FSMContext
4  from aiogram.types import *
5  from aiogram.filters import Command, StateFilter
6  from aiogram.utils.keyboard import InlineKeyboardBuilder
7  from aiogram.fsm.state import State, StatesGroup
8
9
10 # Локальные модули
11 from create_bot import db
12 from utilities.logger import logger
13 from data_base.operations import (
14     doctors_get, doctor_areas_get, dates_get, appointment_add, appointments_get, appointment_get,
15     doctor_area_name_get, appointment_remove
16 )
17
18
19 # Переменные
20 router = Router(name='client')
21
22
23 # Классы
24 class FSMAppointment(StatesGroup): 7 usages  ppjlc
25     full_name = State()
26     doctor_area = State()
27     doctor_area_name = State()
28     doctor_username = State()
29     doctor_full_name = State()
30     builders_date = State()
31     date = State()
32     time = State()
33     first_message = State()

```

Рисунок 22. Код handlers/client.py 1

```

client.py x
37 @router.message(Command(commands=['start'], ignore_case=True))  ± ppljc
38 async def message_start(message: Message, state: FSMContext):
39     try:
40         await state.clear()
41
42         await message.delete()
43
44         builder = InlineKeyboardBuilder()
45         builder.button(text='Записаться 📅', callback_data='add_appointment')
46         builder.button(text='Мои записи 📄', callback_data='list_appointments')
47         builder.adjust(2)
48
49         await message.answer(
50             text=(
51                 '👋 Привет! Вас приветствует бот-помощник Омской Областной Региональной Федеральной Межконтинентальной Больницы.\n\n'
52                 '📄 Я с радостью помогу вам записаться на приём, жмите кнопку ниже!'
53             ),
54             reply_markup=builder.as_markup()
55         )
56
57         logger.info(f'USER={message.from_user.id}, MESSAGE=""')
58     except Exception as e:
59         logger.error(f'USER={message.from_user.id}, MESSAGE="{e}"')

```

Рисунок 23. Код handlers/client.py 2

```

client.py x
62 @router.callback_query(F.data == 'menu_main')  ± ppljc
63 async def callback_menu_main(query: CallbackQuery, state: FSMContext):
64     try:
65         await state.clear()
66
67         await query.answer()
68
69         builder = InlineKeyboardBuilder()
70         builder.button(text='Записаться 📅', callback_data='add_appointment')
71         builder.button(text='Мои записи 📄', callback_data='list_appointments')
72         builder.adjust(2)
73
74         await query.message.edit_text(
75             text=(
76                 '👋 Привет! Вас приветствует бот-помощник Омской Областной Региональной Федеральной Межконтинентальной Больницы.\n\n'
77                 '📄 Я с радостью помогу вам записаться на приём, жмите кнопку ниже!'
78             ),
79             reply_markup=builder.as_markup()
80         )
81
82         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
83     except Exception as e:
84         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 24. Код handlers/client.py 3

```

client.py x
87 @router.callback_query(F.data == 'add_appointment')  ± ppljc
88 async def callback_appointment_add(query: CallbackQuery, state: FSMContext):
89     try:
90         await state.clear()
91
92         await query.answer()
93
94         builder = InlineKeyboardBuilder()
95         builder.button(text='Назад 🏠', callback_data='menu_main')
96
97         await query.message.edit_text(
98             text='👋 Отправьте своё ФИО:',
99             reply_markup=builder.as_markup()
100         )
101
102         await state.set_state(FSMAppointment.full_name)
103         await state.update_data(first_message=query.message)
104
105         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
106     except Exception as e:
107         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 25. Код handlers/client.py 4

```

client.py x
110 @router.message(StateFilter(FSMAppointment.full_name))  # ppljc
111 async def message_appointment_full_name(message: Message, state: FSMContext):
112     try:
113         data = await state.get_data()
114         first_message = data['first_message']
115
116         await message.delete()
117
118         reply_markup = await doctor_areas_get(db=db)
119
120         await first_message.edit_text(
121             text=f'👤 {message.text}, выберите направление, на которое хотите попасть на приём:',
122             reply_markup=reply_markup
123         )
124
125         await state.set_state(FSMAppointment.doctor_area)
126         await state.update_data(full_name=message.text)
127
128         logger.info(f'USER={message.from_user.id}, MESSAGE="full_name={message.text}"')
129     except Exception as e:
130         logger.error(f'USER={message.from_user.id}, MESSAGE="{e}"')

```

Рисунок 26. Код handlers/client.py 5

```

client.py x
133 @router.callback_query(F.data == 'appointment_full_name')  # ppljc
134 async def callback_appointment_full_name(query: CallbackQuery, state: FSMContext):
135     try:
136         data = await state.get_data()
137         first_message = data['first_message']
138         full_name = data['full_name']
139
140         await query.answer()
141
142         reply_markup = await doctor_areas_get(db=db)
143
144         await first_message.edit_text(
145             text=f'👤 {full_name}, выберите направление, на которое хотите попасть на приём:',
146             reply_markup=reply_markup
147         )
148
149         await state.set_state(FSMAppointment.doctor_area)
150
151         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
152     except Exception as e:
153         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 27. Код handlers/client.py 6

```

client.py x
156 @router.callback_query(F.data.startswith('appointment_doctor_area')) ± ppijc
157 async def callback_appointment_doctor_area(query: CallbackQuery, state: FSMContext):
158     try:
159         data = await state.get_data()
160         full_name = data['full_name']
161
162         await query.answer()
163         doctor_area = query.data.split('|')[1]
164
165         reply_markup, doctor_area_name = await doctors_get(db=db, doctor_area=doctor_area)
166
167         await query.message.edit_text(
168             text=f'👤 {full_name}, выберите врача по направлению «{doctor_area_name}», к которому хотите попасть на приём:',
169             reply_markup=reply_markup
170         )
171
172         await state.set_state(FSMAppointment.doctor_username)
173         await state.update_data(
174             doctor_area=doctor_area,
175             doctor_area_name=doctor_area_name
176         )
177
178         logger.info(f'USER={query.from_user.id}, MESSAGE="doctor_area={doctor_area}"')
179     except Exception as e:
180         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 28. Код handlers/client.py 7

```

client.py x
183 @router.callback_query(F.data.startswith('appointment_doctor_username')) ± ppijc
184 async def callback_appointment_doctor_username(query: CallbackQuery, state: FSMContext):
185     try:
186         data = await state.get_data()
187         full_name = data['full_name']
188         doctor_area = data['doctor_area']
189         doctor_area_name = data['doctor_area_name']
190
191         await query.answer()
192         doctor_username = query.data.split('|')[1]
193
194         reply_markup, builders_date, doctor_full_name = await dates_get(db=db, doctor_area=doctor_area, doctor_username=doctor_username)
195
196         await query.message.edit_text(
197             text=f'📅 {full_name}, на какую дату вы хотите записаться к врачу «{doctor_full_name}» по направлению «{doctor_area_name}»: ',
198             reply_markup=reply_markup
199         )
200
201         await state.set_state(FSMAppointment.date)
202         await state.update_data(
203             doctor_username=doctor_username,
204             doctor_full_name=doctor_full_name,
205             builders_date=builders_date
206         )
207
208         logger.info(f'USER={query.from_user.id}, MESSAGE="doctor_username={doctor_username}"')
209     except Exception as e:
210         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 29. Код handlers/client.py 8

```

client.py x
213 @router.callback_query(F.data.startswith('appointment_date'))  # ppjlc
214 async def callback_appointment_date(query: CallbackQuery, state: FSMContext):
215     try:
216         data = await state.get_data()
217         full_name = data['full_name']
218         doctor_full_name = data['doctor_full_name']
219         doctor_area_name = data['doctor_area_name']
220         builders_date = data['builders_date']
221
222         await query.answer()
223         date = query.data.split('|')[1]
224
225         await query.message.edit_text(
226             text=f'🕒 {full_name}, на какое время вы хотите записаться к врачу «{doctor_full_name}» по направлению «{doctor_area_name}» на {date}:',
227             reply_markup=builders_date[date]
228         )
229
230         await state.set_state(FSMAppointment.time)
231         await state.update_data(date=date)
232
233         logger.info(f'USER={query.from_user.id}, MESSAGE="date={query.data}"')
234     except Exception as e:
235         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 30. Код handlers/client.py 9

```

client.py x
238 @router.callback_query(F.data.startswith('appointment_time'))  # ppjlc
239 async def callback_appointment_time(query: CallbackQuery, state: FSMContext):
240     try:
241         data = await state.get_data()
242         full_name = data['full_name']
243         doctor_area = data['doctor_area']
244         doctor_area_name = data['doctor_area_name']
245         doctor_username = data['doctor_username']
246         doctor_full_name = data['doctor_full_name']
247
248         await query.answer()
249         date = query.data.split('|')[1]
250         time = query.data.split('|')[2]
251
252         await appointment_add(
253             db=db,
254             user_id=query.from_user.id,
255             date=date,
256             time=time,
257             doctor_area=doctor_area,
258             doctor_username=doctor_username
259         )
260
261         builder = InlineKeyboardBuilder()
262         builder.button(text='Назад 🏠', callback_data='menu_main')
263
264         await query.message.edit_text(
265             text=f'✅ {full_name}, вы успешно записаны к «{doctor_full_name}» на направление «{doctor_area_name}» на {date} в {time}!',
266             reply_markup=builder.as_markup()
267         )
268
269         await state.clear()
270
271         logger.info(f'USER={query.from_user.id}, MESSAGE="time={time}"')
272     except Exception as e:
273         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 31. Код handlers/client.py 10

```

client.py x
276 @router.callback_query(F.data == 'list_appointments') 1 usage 1 ppjlc
277 async def callback_appointment_list(query: CallbackQuery):
278     try:
279         await query.answer()
280
281         text, reply_markup = await appointments_get(db=db, user_id=query.from_user.id)
282
283         await query.message.edit_text(
284             text=text,
285             reply_markup=reply_markup
286         )
287
288         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
289     except Exception as e:
290         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')
291
292
293 @router.callback_query(F.data.startswith('edit_appointment')) 1 ppjlc
294 async def callback_appointment_edit(query: CallbackQuery):
295     try:
296         await query.answer()
297         date = query.data.split('|')[1]
298         time = query.data.split('|')[2]
299         doctor_area = query.data.split('|')[3]
300         doctor_username = query.data.split('|')[4]
301
302         text, reply_markup = await appointment_get(db=db, date=date, time=time, doctor_area=doctor_area, doctor_username=doctor_username)
303
304         await query.message.edit_text(
305             text=text,
306             reply_markup=reply_markup
307         )
308
309         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
310     except Exception as e:
311         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 32. Код handlers/client.py 11

```

client.py x
314 @router.callback_query(F.data.startswith('remove_appointment')) 1 ppjlc
315 async def callback_appointment_remove(query: CallbackQuery):
316     try:
317         await query.answer()
318         date = query.data.split('|')[1]
319         time = query.data.split('|')[2]
320         doctor_area = query.data.split('|')[3]
321         doctor_username = query.data.split('|')[4]
322
323         doctor_area_name = await doctor_area_name_get(db=db, doctor_area=doctor_area)
324
325         builder = InlineKeyboardBuilder()
326         builder.button(text='Да 🗑️', callback_data=f'confirm_remove_appointment|{date}|{time}|{doctor_area}|{doctor_username}')
327         builder.button(text='Назад 🏠', callback_data=f'edit_appointment|{date}|{time}|{doctor_area}|{doctor_username}')
328         builder.adjust(2)
329
330         await query.message.edit_text(
331             text=f' ? Вы уверены, что хотите удалить запись на {date} в {time} к {doctor_area_name}?',
332             reply_markup=builder.as_markup()
333         )
334
335         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
336     except Exception as e:
337         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 33. Код handlers/client.py 12

```

340 @router.callback_query(F.data.startswith('confirm_remove_appointment')) 1 ppjlc
341 async def callback_appointment_confirm_remove(query: CallbackQuery):
342     try:
343         await query.answer()
344         date = query.data.split('|')[1]
345         time = query.data.split('|')[2]
346         doctor_area = query.data.split('|')[3]
347         doctor_username = query.data.split('|')[4]
348
349         await appointment_remove(db=db, user_id=query.from_user.id, date=date, time=time, doctor_area=doctor_area, doctor_username=doctor_username)
350
351         await callback_appointment_list(query=query)
352
353         logger.info(f'USER={query.from_user.id}, MESSAGE=""')
354     except Exception as e:
355         logger.error(f'USER={query.from_user.id}, MESSAGE="{e}"')

```

Рисунок 34. Код handlers/client.py 13

Нотация к коду

При запуске бота инициализируется база данных. Она содержит базу данных пользователей, записей к докторам, направления докторов и таблицу со всеми докторами. При любом запросе пользователь добавляется в базу данных. Функции базы данных созданы с защитой от SQL инъекции.

Процесс записи в больницу производится с помощью FSM машины и её различных состояний, в которых и хранятся собираемые данные.

Все кнопки назад в боте ведут на предыдущий шаг, либо в главное меню.

Результат работы

Красным обозначены места, куда производились нажатия.

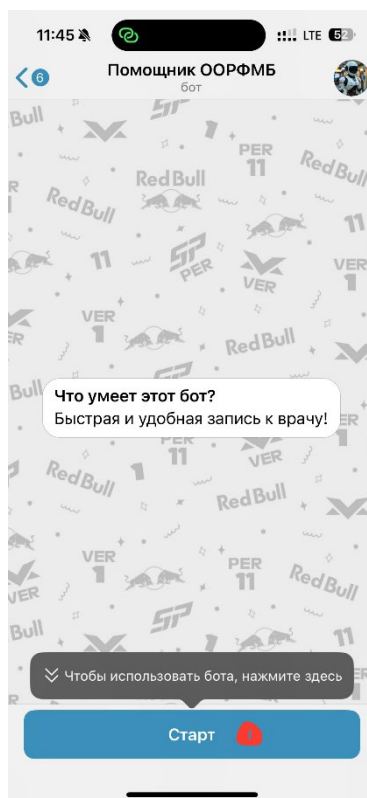


Рисунок 35. Результат работы 1

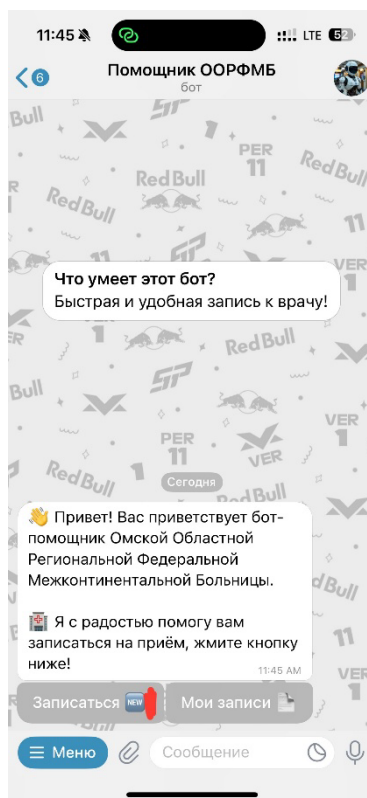


Рисунок 36. Результат работы 2

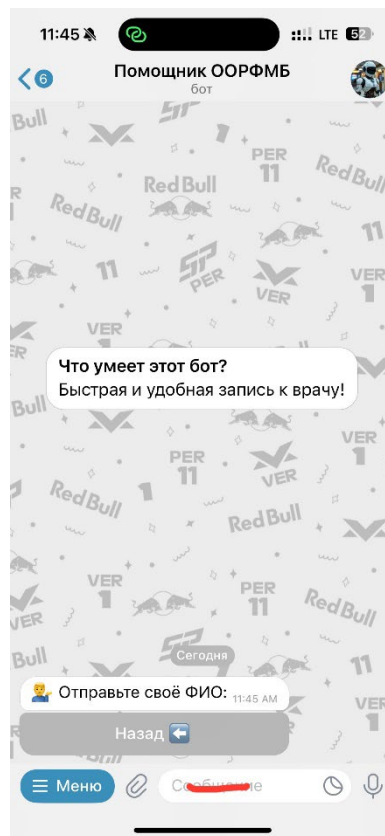


Рисунок 37. Результат работы 3

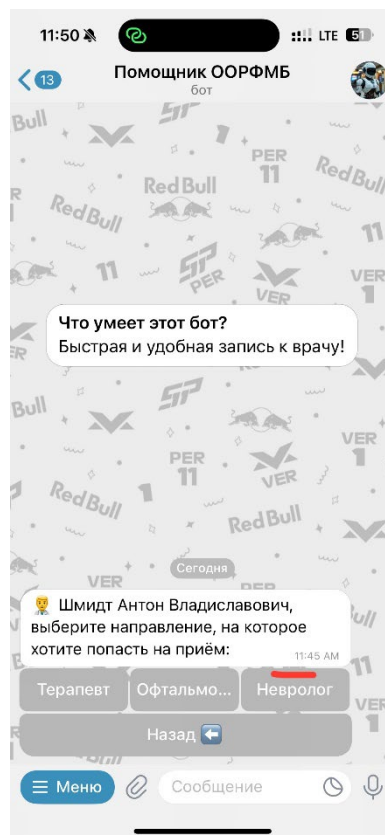


Рисунок 38. Результат работы 4

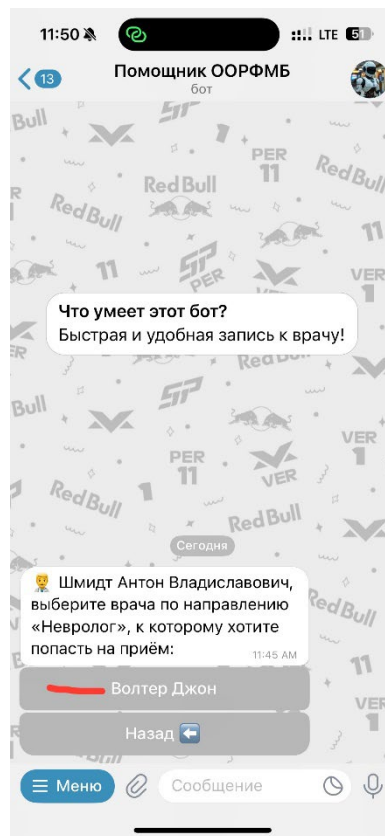


Рисунок 39. Результат работы 5

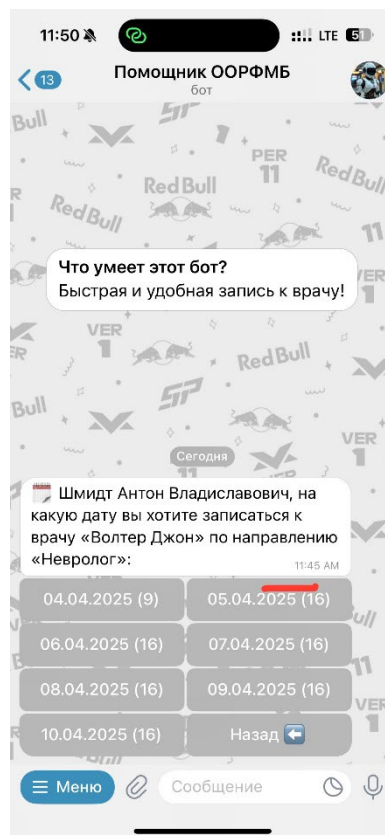


Рисунок 40. Результат работы 6

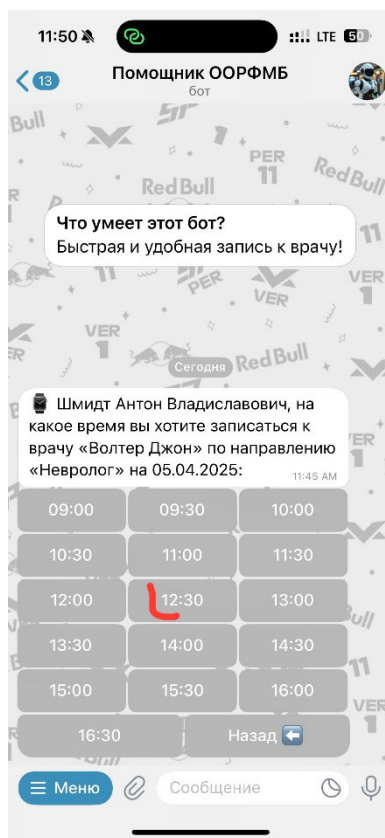


Рисунок 41. Результат работы 7

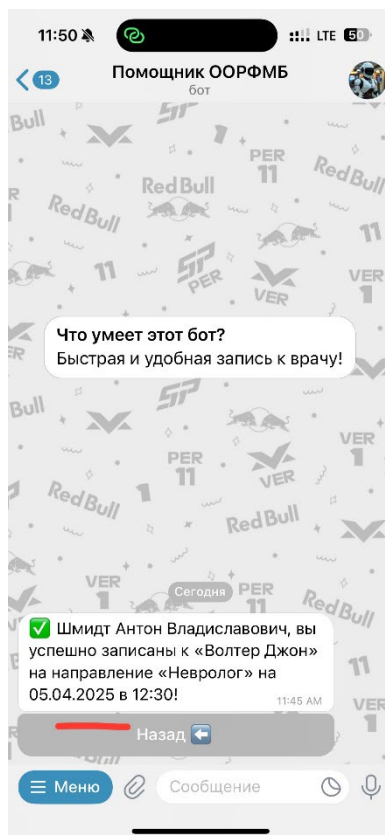


Рисунок 42. Результат работы 8

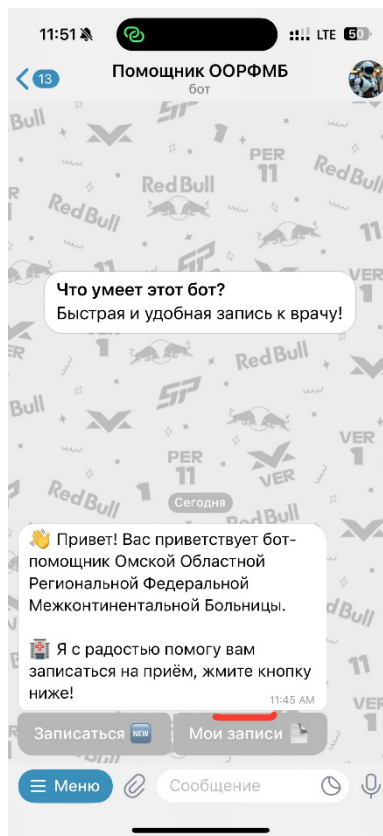


Рисунок 43. Результат работы 9



Рисунок 44. Результат работы 10

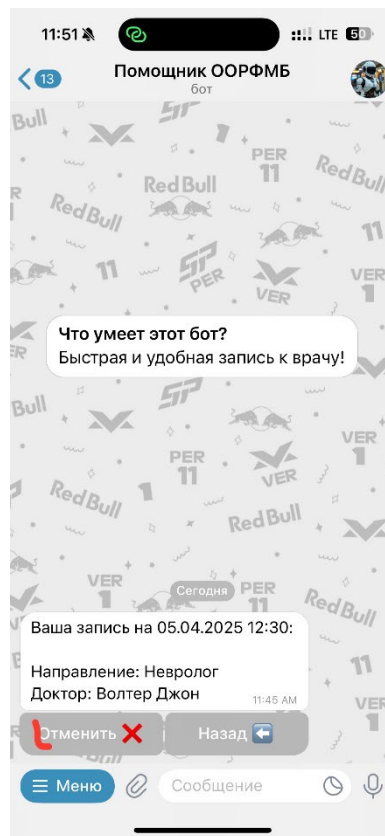


Рисунок 45. Результат работы 11



Рисунок 46. Результат работы 12



Рисунок 47. Результат работы 13

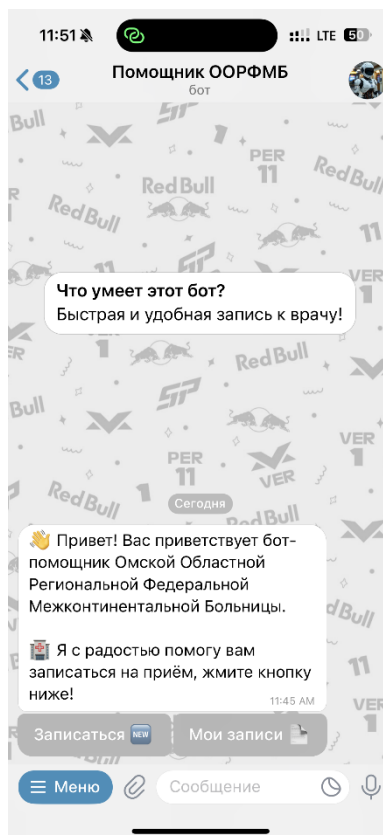


Рисунок 48. Результат работы 14

Заключение

В результате проделанной работы был написан телеграм-бот, который позволяет записаться в больницу с указанием ФИО, направления, врача, даты и времени.

Ссылка на репозиторий: ppljc/opd-lab-work-2.